Information Security

BERR 3123

SEM 1 2025/2026

Damn Venerable Web App **Lab X1** (DVWA 01)

	Lab 1 (week #1 to #3)	Lab 2 (week #)	Lab 3 (week #)
Lab (short report)	DVWA X1 (password login brute force)	DVWA X2 (Design flaw of reCAPTCHA)	DVWA X3 (SQL Injection)
Lab (long report)	Wireshark/tcpdump analysis		

With samples

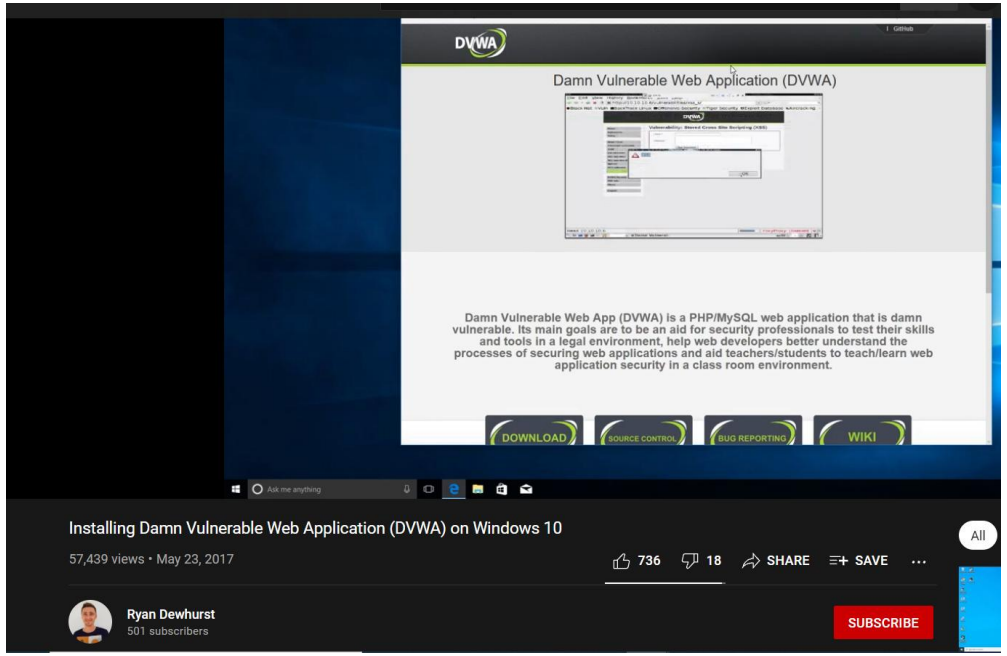
Lab (DVWA)

https://github.com/digininja/DVWA

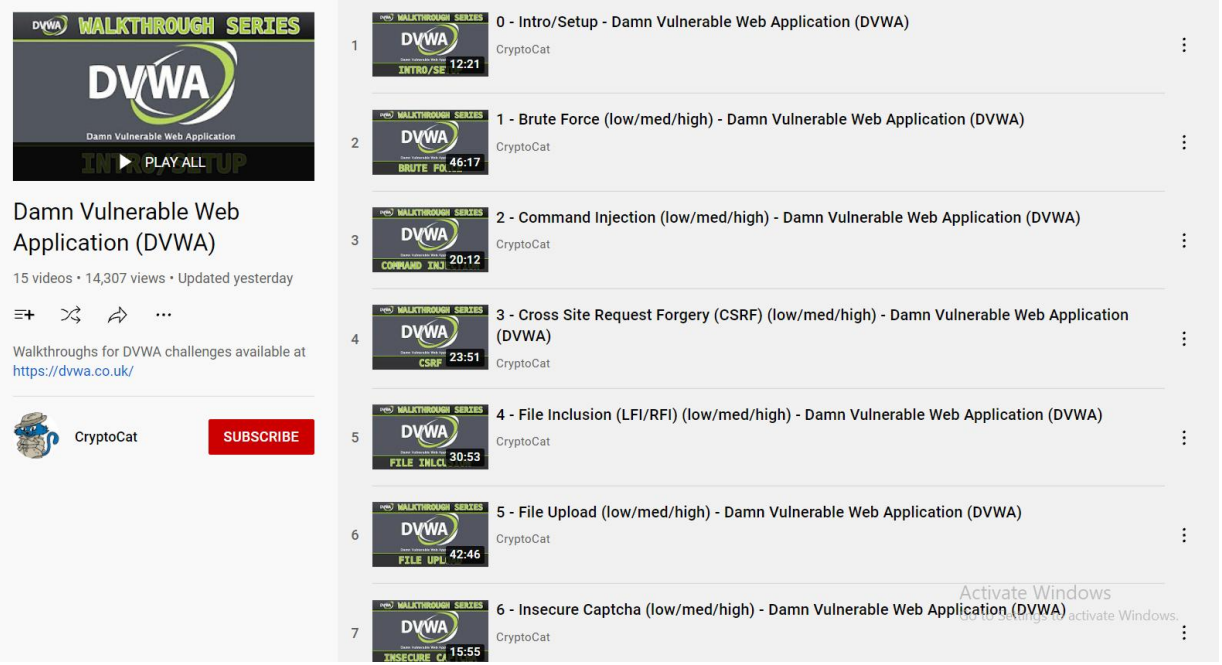
Administrator: Windows PowerShell

```
PS C:\WINDOWS\system32> wsl -l -v
  NAME                   STATE           VERSION
* Ubuntu-18.04           Stopped         2
  kali-linux             Stopped         2
```

https://www.kali.org/docs/wsl/win-kex/



https://www.youtube.com/watch?v=cak2lQvBRAo



https://www.youtube.com/watch?v=GmWQ1VIjd2U&list=PLHUKi1UlEgOJLPSFZaFKMoexpM6qhOb4Q

Lab report layout for each vulnerability

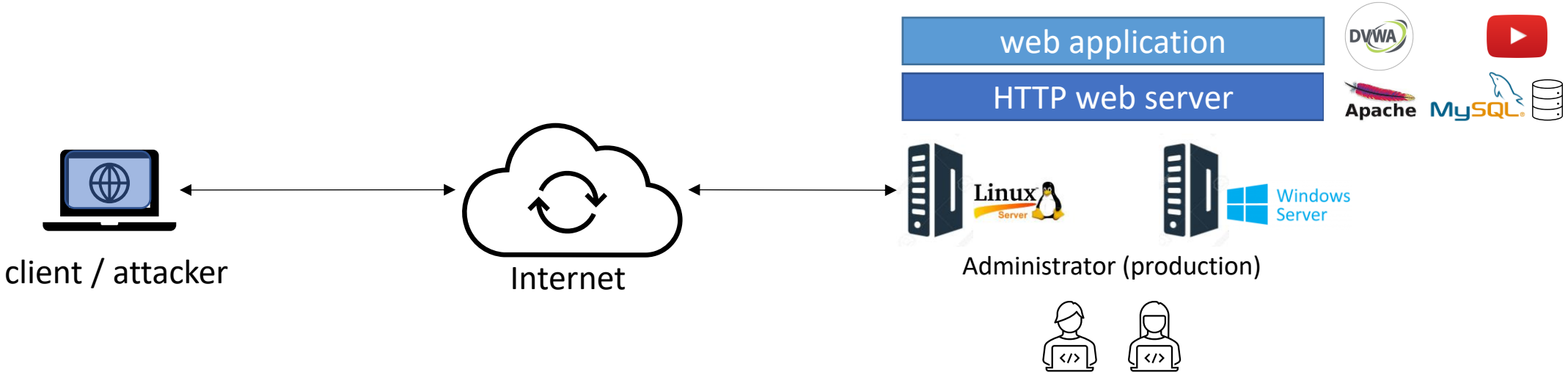
a) What is the **vulnerability** and what will happen to the **CIA** when the vulnerability is exploited?

b) How the vulnerability can be exploited? What is the **attack surface**?

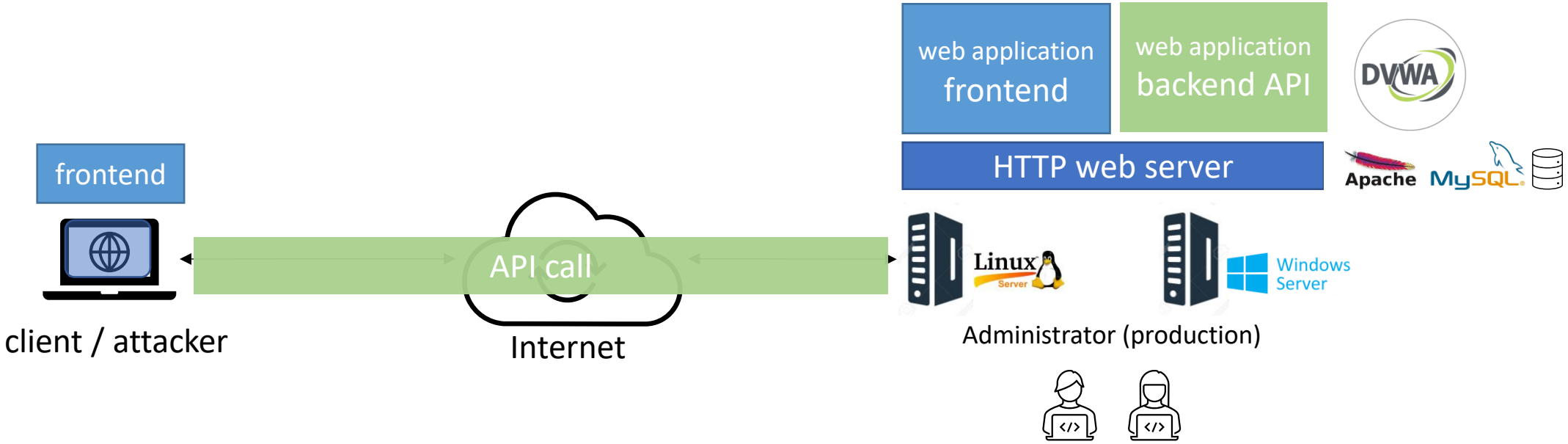
c) Wireshark network adapter(s) used, filtering used, and flow graph analysis of the attack.

d) Result and discussion from the captured wireshark packets.

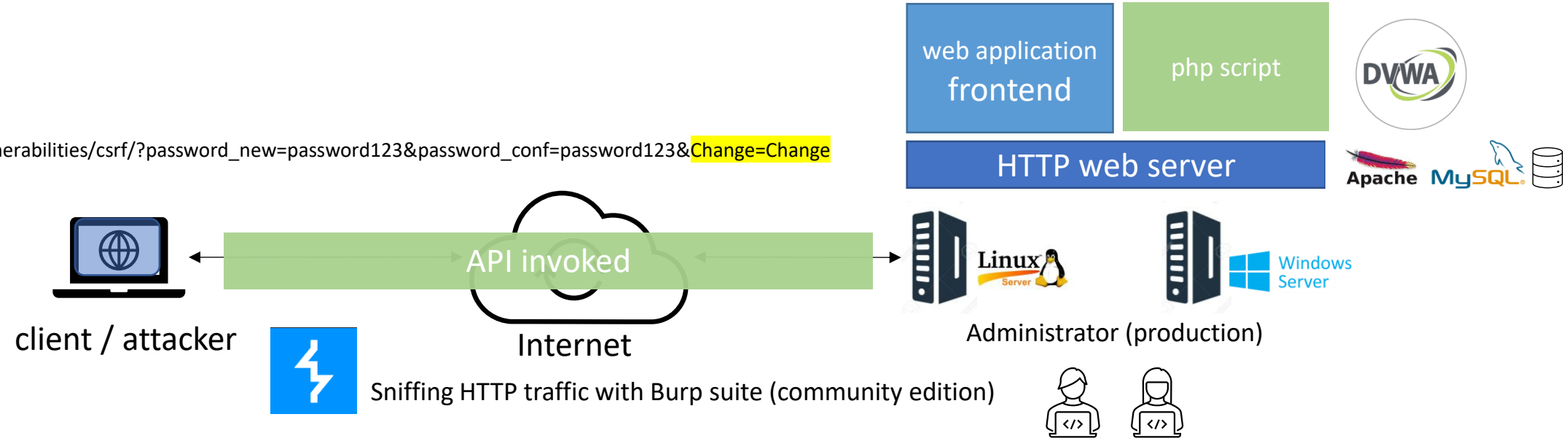
Lab activity	Assets	Threat Agent	Attack surface	Attack Goal	Impact	Controls (Mitigation)
DVWA Lab 1 Password Brute Force Attack	User credential	Unauthorized internal/external user launching brute force attack	Weak credential at the web app server. Password login web app design flaw.	Obtained user credential	Compromised of user credential and details in the database	Mid: suspend all user login activities by 3 seconds sleep upon authentication failure (scarify the availability) High: … Your opinion:…



Web app can be developed with different programming language (perl, php, java, HTML5) or framework (react, angular). Depend on the common engineering skill set or preferences, the web server could be running windows or Linux.



Application Interface (API) are designed to allow Internet browser using HTTP GET and POST to interact with the web application running in the server



The php script running at the web server to handle the API Change will change the admin password when the API is invoked.

```php
<?php
if( isset( $_GET[ 'Change' ] ) ) {
   // Get input
   $pass_new  = $_GET[ 'password_new' ];
   $pass_conf = $_GET[ 'password_conf' ];

   // Do the passwords match?
   if( $pass_new == $pass_conf ) {
      // They do!
      $pass_new = ((isset($GLOBALS["___mysqli_ston"]) && is_object($GLOBALS["___mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS
["___mysqli_ston"],  $pass_new ) : ((trigger_error("[MySQLConverterToo] Fix the mysql_escape_string() call! This code does not work.", E_USE
R_ERROR)) ? "" : ""));
      $pass_new = md5( $pass_new );

      // Update the database
      $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser() . "';";
      $result = mysqli_query($GLOBALS["___mysqli_ston"],  $insert ) or die( '<pre>' . ((is_object($GLOBALS["___mysqli_ston"])) ? mysqli_error($
GLOBALS["___mysqli_ston"]) : (($___mysqli_res = mysqli_connect_error()) ? $___mysqli_res : false)) . '</pre>' );

      // Feedback for the user
      echo "<pre>Password Changed.</pre>";
   }
   else {
      // Issue with passwords matching
      echo "<pre>Passwords did not match.</pre>";
   }

   ((is_null($___mysqli_res = mysqli_close($GLOBALS["___mysqli_ston"]))) ? false : $___mysqli_res);
}
?>
```

DVWA/vulnerabilities/csrf/?password_new=password123&password_conf=password123&Change=Change

The php script running at the web server to handle the API Change will change the password of the current user (returned by function dvwaCurrentUser()) when the API Change is invoked.

Vulnerability: Cross Site Request ×

172.22.173.114/DVWA/vulnerabilities/csrf/?password_new=password123&password_conf=password123&Change=Change#

DVWA

- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection
- SQL Injection (Blind)

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password: ••••••••

Confirm new password:

Change

Password Changed.

API involved when button is pressed

DVWA/vulnerabilities/csrf/?password_new=password123&password_conf=password123&Change=Change

Request

Pretty | Raw | Hex

```
1  GET /DVWA/vulnerabilities/csrf/?password_new=password123&password_conf=password123&Change=
   Change HTTP/1.1
2  Host: 172.22.173.114
3  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:94.0) Gecko/20100101 Firefox/94.0
4  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
5  Accept-Language: en-US,en;q=0.5
6  Accept-Encoding: gzip, deflate
7  Connection: close
8  Referer: http://172.22.173.114/DVWA/vulnerabilities/csrf/
9  Cookie: security=low; PHPSESSID=ae3413rd1960bbe63hc4selk21
10 Upgrade-Insecure-Requests: 1
11
12
```

Whoever is the current user already login to the DVWA server, the password will be changed!!!

Response

Pretty | Raw | Hex | Render

```
1  HTTP/1.1 200 OK
2  Date: Thu, 09 Dec 2021 00:29:15 GMT
3  Server: Apache/2.4.29 (Ubuntu)
4  Expires: Tue, 23 Jun 2009 12:00:00 GMT
5  Cache-Control: no-cache, must-revalidate
6  Pragma: no-cache
7  Vary: Accept-Encoding
8  Content-Length: 4494
9  Connection: close
10 Content-Type: text/html;charset=utf-8
11
12 <!DOCTYPE html>
13
14 <html lang="en-GB">
15
16   <head>
17     <meta http-equiv="Content-Type" content="text/html;
       charset=UTF-8" />
18
19     <title>
         Vulnerability: Cross Site Request Forgery (CSRF) :: Damn
```

INSPECTOR

Request Attributes

Protocol: HTTP/1 | HTTP/2

ATTRIBUTE	VALUE
Method	GET
Path	/DVWA/vulnerabilities...

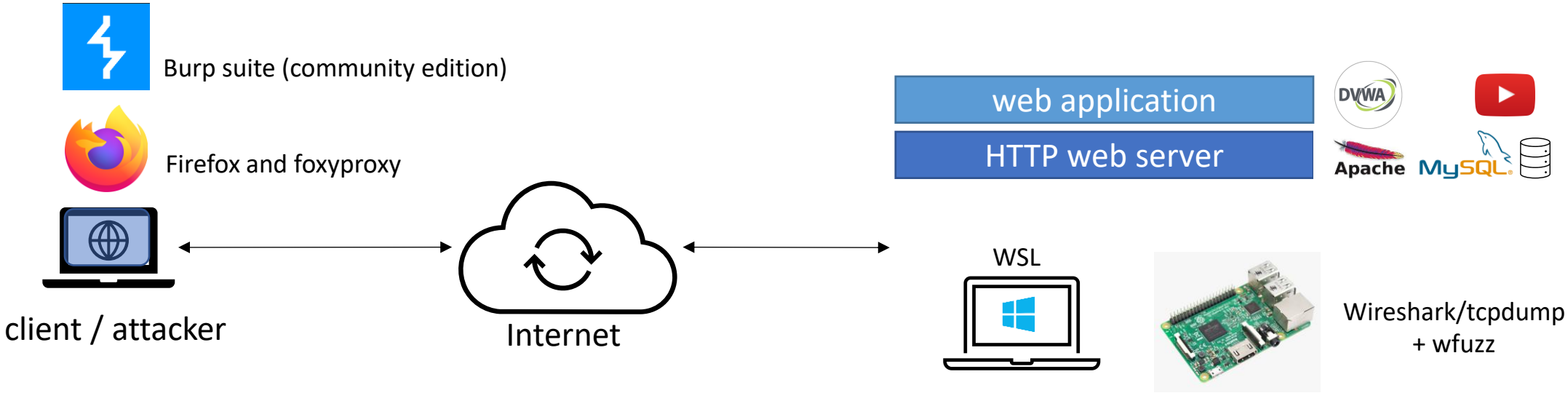
Query Parameters (3)

NAME	VALUE
password_new	password123
password_conf	password123
Change	Change

Request Cookies (2)

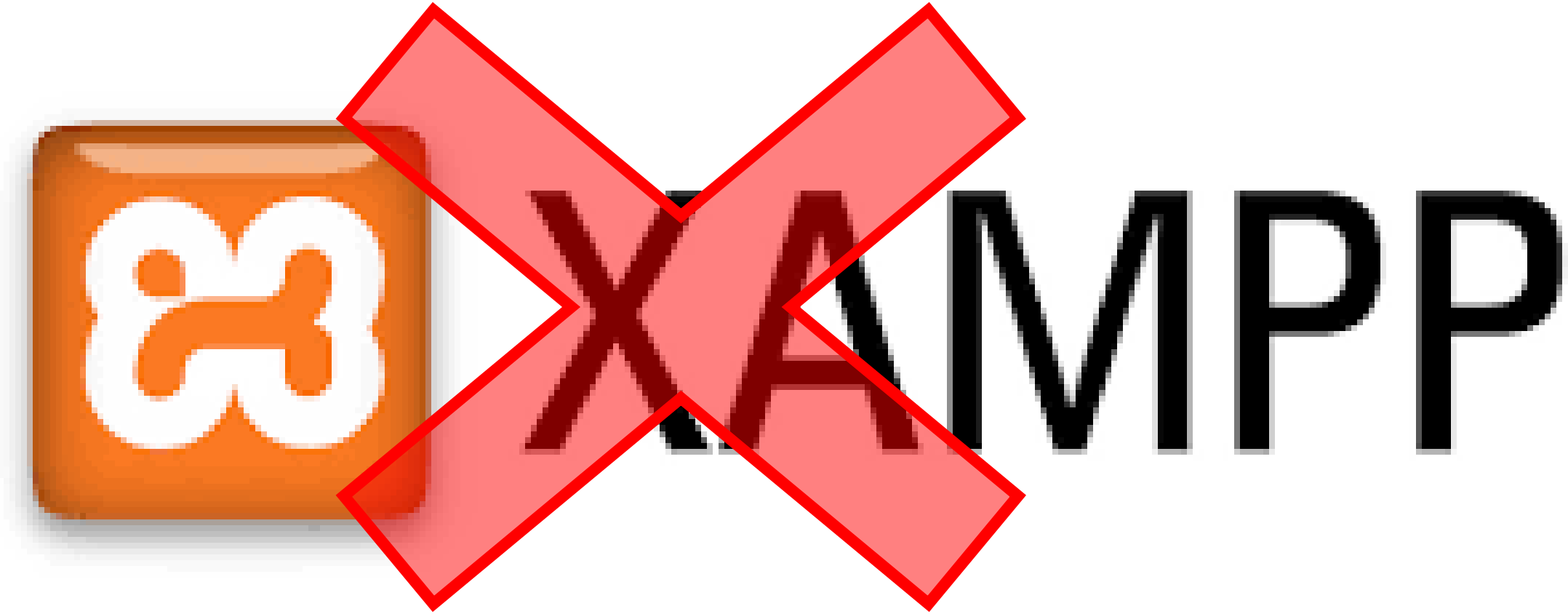
Step 0: Prepare the hardware, OS and software

https://www.youtube.com/watch?v=GmWQ1VIjd2U&list=PLHUKi1UlEgOJLPSFZaFKMoexpM6qhOb4Q&index=1



Suggestion #1: WSL2 + Ubuntu 20.04 and install DVWA
https://www.ceos3c.com/wsl-2/install-wsl2-with-windows-terminal/
Suggestion #2: Get a RPi and install DVWA
Suggestion #3: Do whatever you can

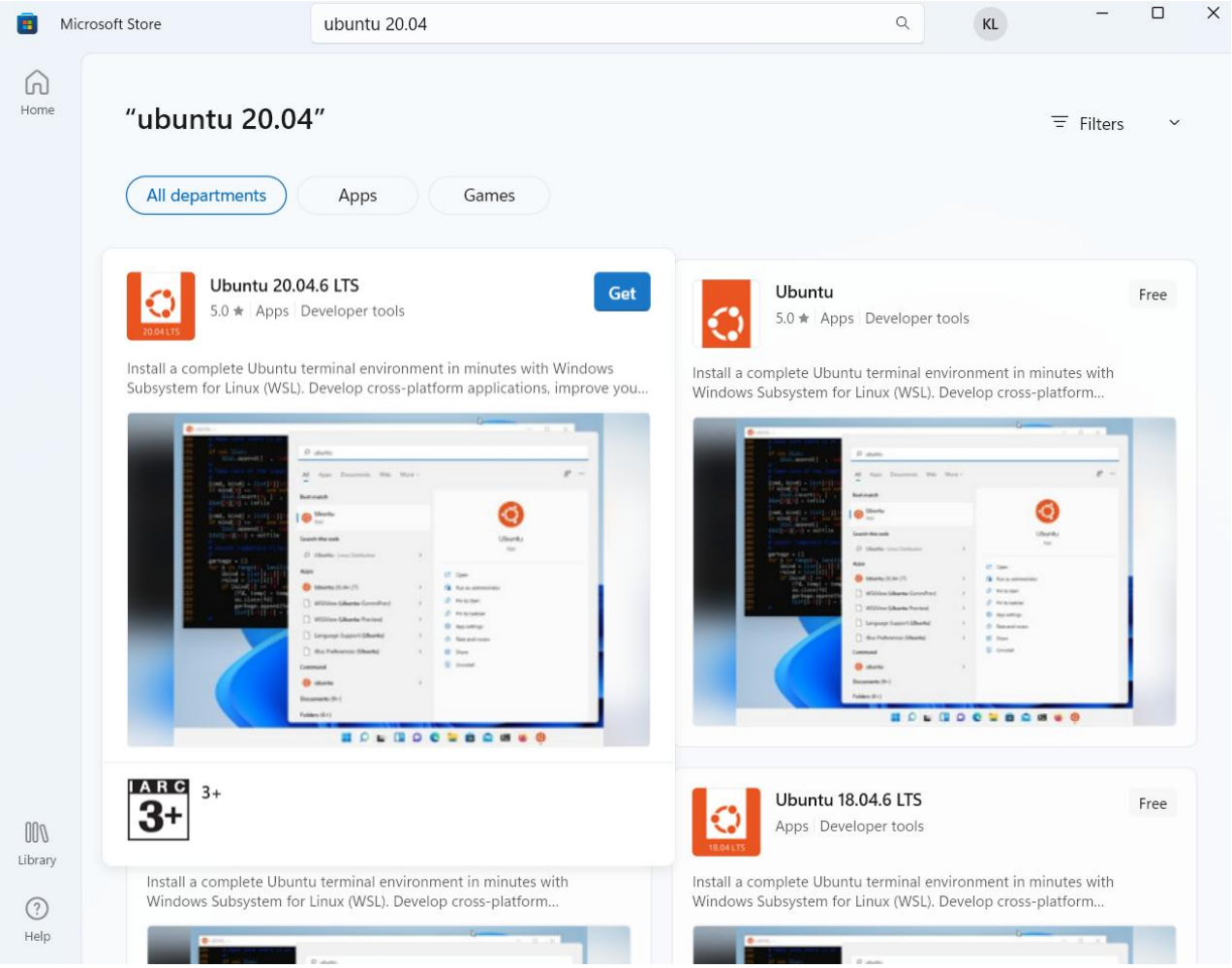
Install wfuzz in ubuntu/RPi with sudo apt-get install python-setuptools wfuzz



Strictly no XAMPP!!!! Please redo with WSL2 Ubuntu 20.04

Installing DVWA into the WSL2 Ubuntu 20.04

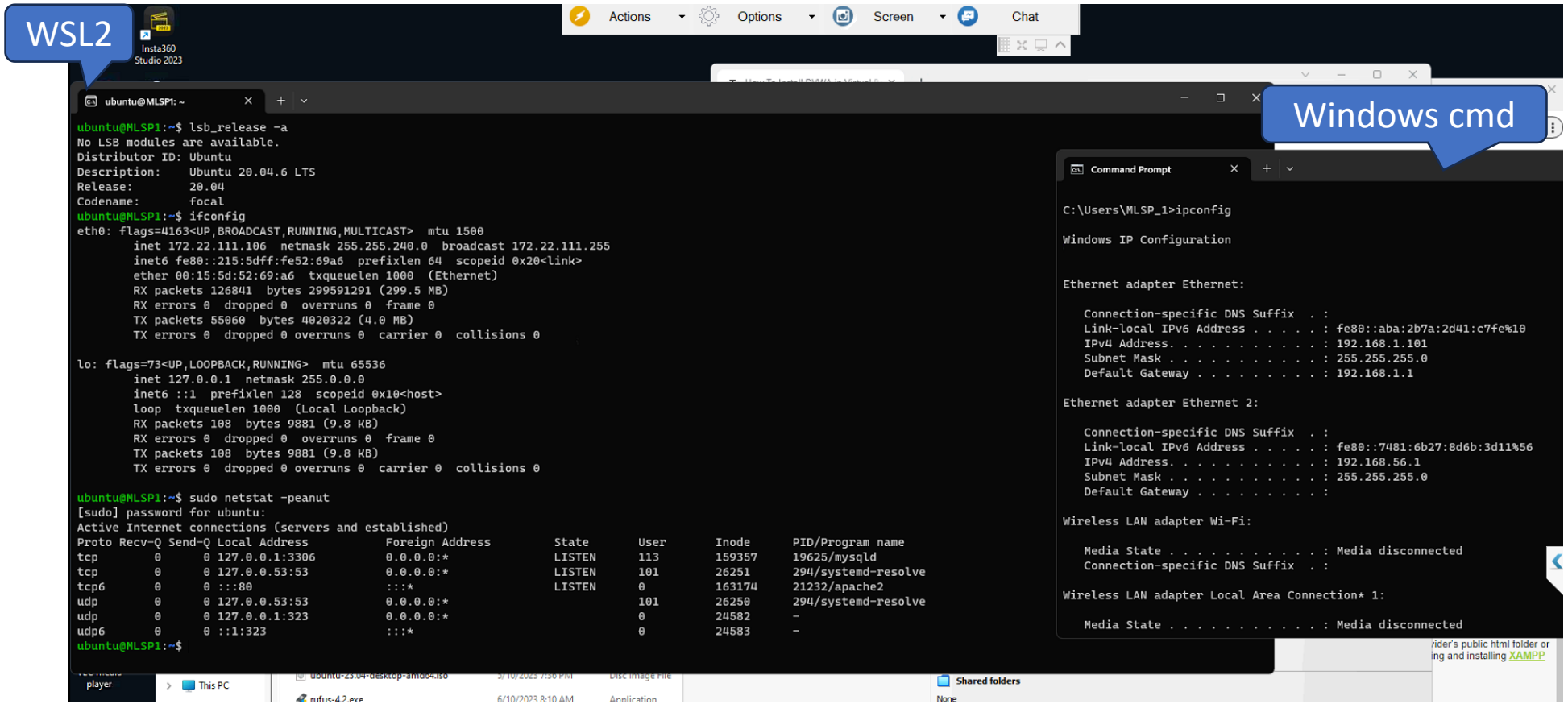
Step #0 Install Ubuntu 20.04 from Windows Store (you might want to google to find out how to properly configure your Windows WSL2)



Installing DVWA into the Ubuntu

https://github.com/digininja/DVWA

Step #1 Start the WSL2 Ubuntu 20.04 and get yourself familiar with the command line interface



Installing DVWA into the Ubuntu

https://github.com/digininja/DVWA

Step #2 Update software repositories with apt update and proceed to install the required apache http server and database server needed by DVWA

apt update
apt install -y apache2 mariadb-server mariadb-client php php-mysqli php-gd libapache2-mod-php

```
ubuntu@MLSP1:~$ sudo netstat -peanut
[sudo] password for ubuntu:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User       Inode      PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      113        159357     19625/mysqld
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      101        26251      294/systemd-resolve
tcp6       0      0 :::80                   :::*                    LISTEN      0          163174     21232/apache2
udp        0      0 127.0.0.53:53           0.0.0.0:*                           101        26250      294/systemd-resolve
udp        0      0 127.0.0.1:323           0.0.0.0:*                           0          24582      -
udp6       0      0 ::1:323                 :::*                                0          24583      -
ubuntu@MLSP1:~$
```

Sql database is listening port 3306

Apache http server is listening port 80

What’s the different between listening to “127.0.0.1” and “::” ?

```
ubuntu@MLSP1:~$ sudo netstat -peanut
[sudo] password for ubuntu:
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User       Inode      PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      113        159357     19625/mysqld
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      101        26251      294/systemd-resolve
tcp6       0      0 :::80                   :::*                    LISTEN      0          163174     21232/apache2
udp        0      0 127.0.0.53:53           0.0.0.0:*                           101        26250      294/systemd-resolve
udp        0      0 127.0.0.1:323           0.0.0.0:*                           0          24582      -
udp6       0      0 ::1:323                 :::*                                0          24583      -
ubuntu@MLSP1:~$
```

Sql database is listening port 3306

Apache http server is listening port 80

Installing DVWA into the Ubuntu

https://github.com/digininja/DVWA

Step #3 git clone the DVWA github repository into the WSL2 Ubuntu 20.04

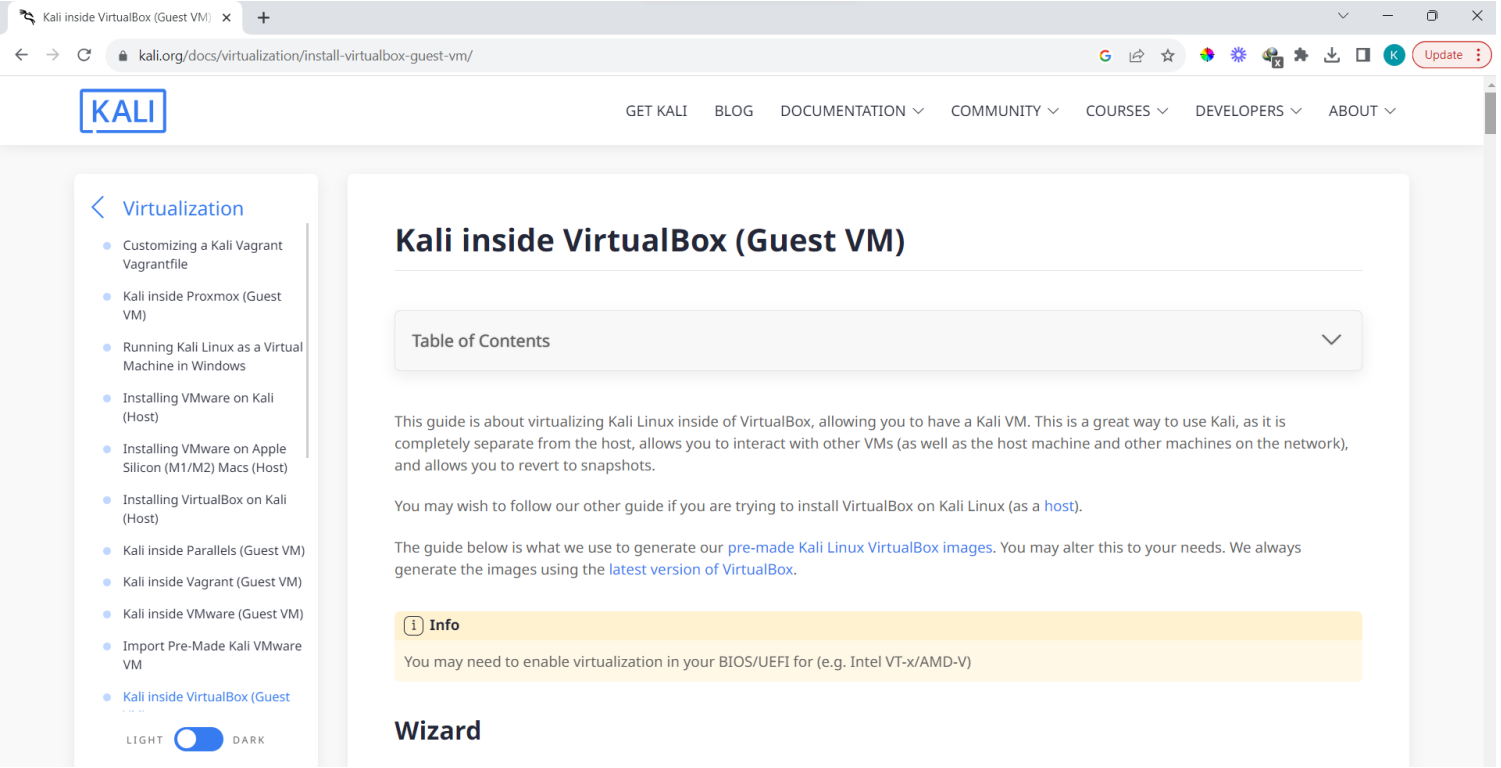
git clone https://github.com/digininja/DVWA

a folder name “DVWA” should be created.

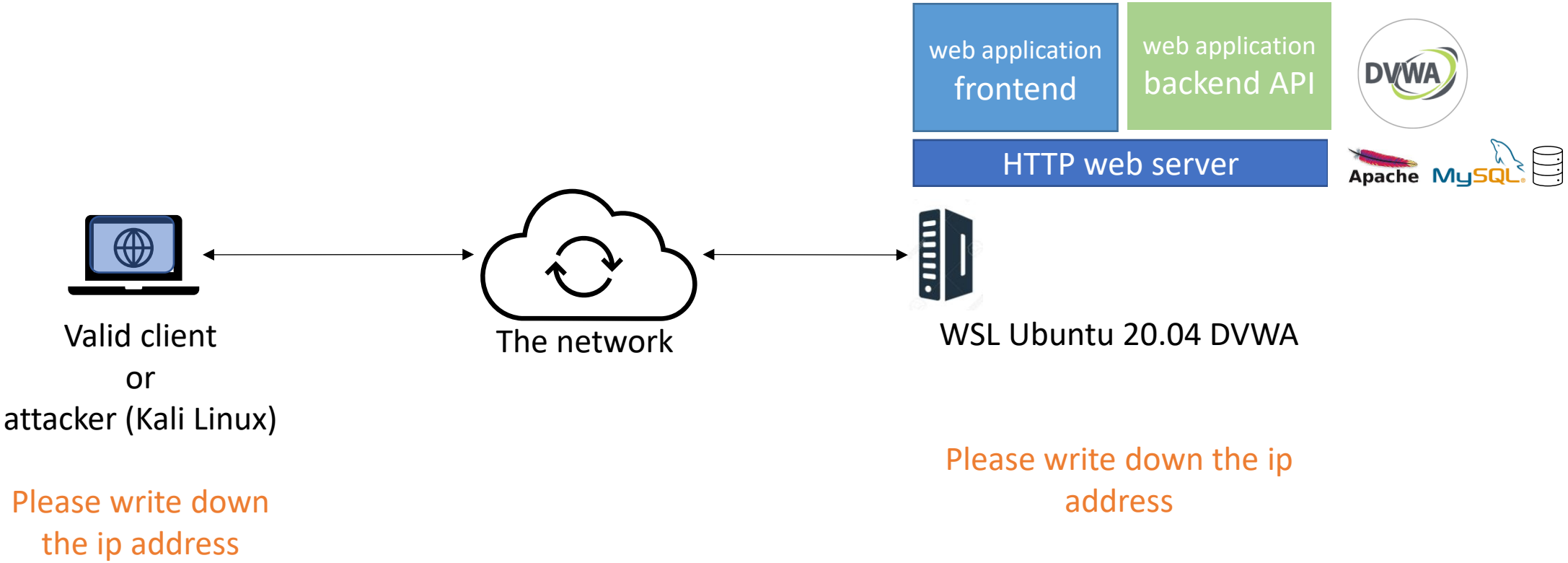
By default, the apache HTTP server uses directory /var/www/html to serve all the web pages. You can try to setup the DVWA in /var/www/html by following the video (https://www.youtube.com/watch?v=GmWQ1VIjd2U&list=PLHUKi1UlEgOJLPSFZaFKMoexpM6qhOb4Q&index=1).

Installing Kali Linux (Virtualbox)

https://www.kali.org/docs/virtualization/install-virtualbox-guest-vm/

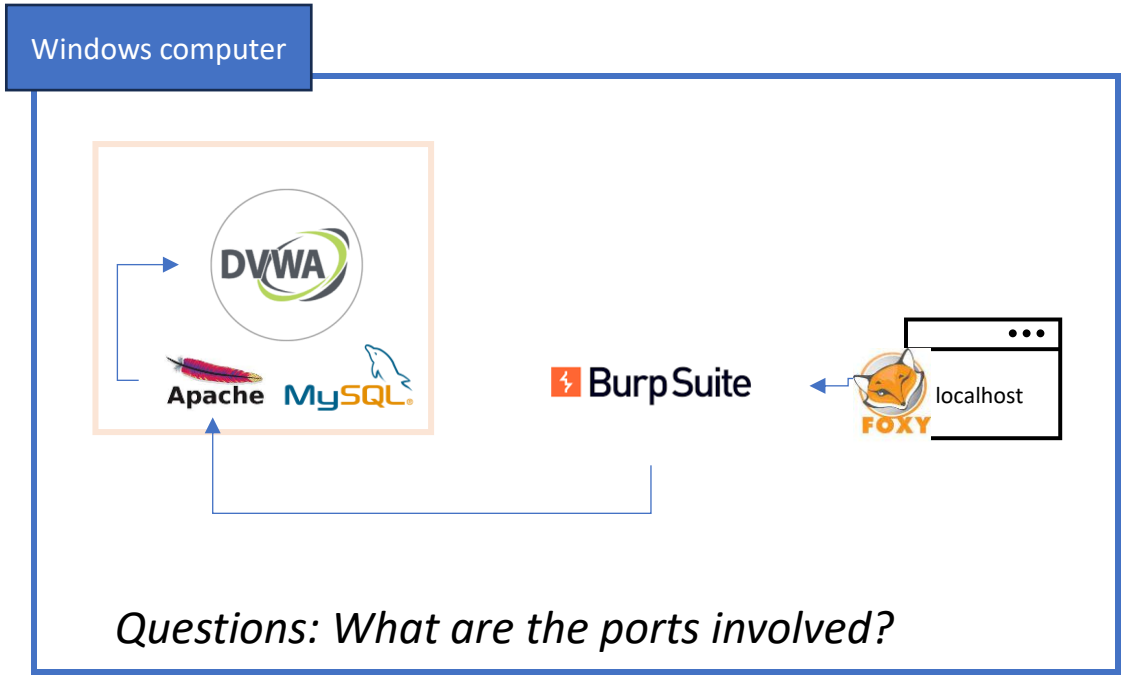


The complete setup should be as follow:



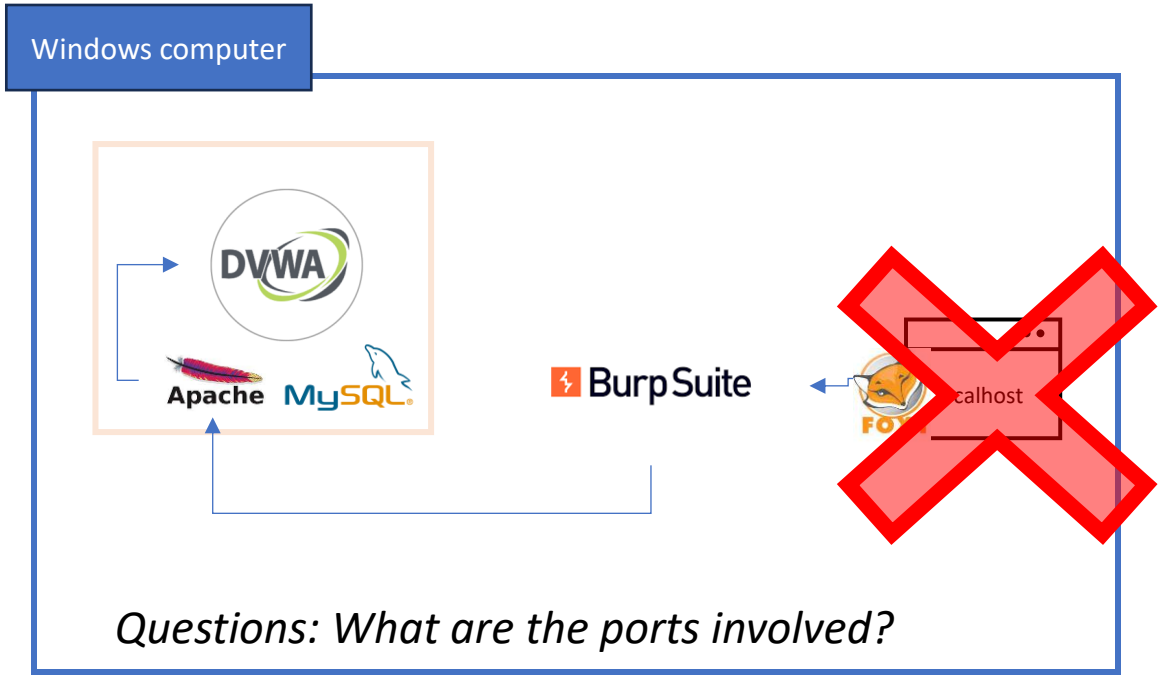
To begin with (simple pure windows setup)

attack over the windows localhost with BurpSuite running in Windows



To begin with (simple pure windows setup)

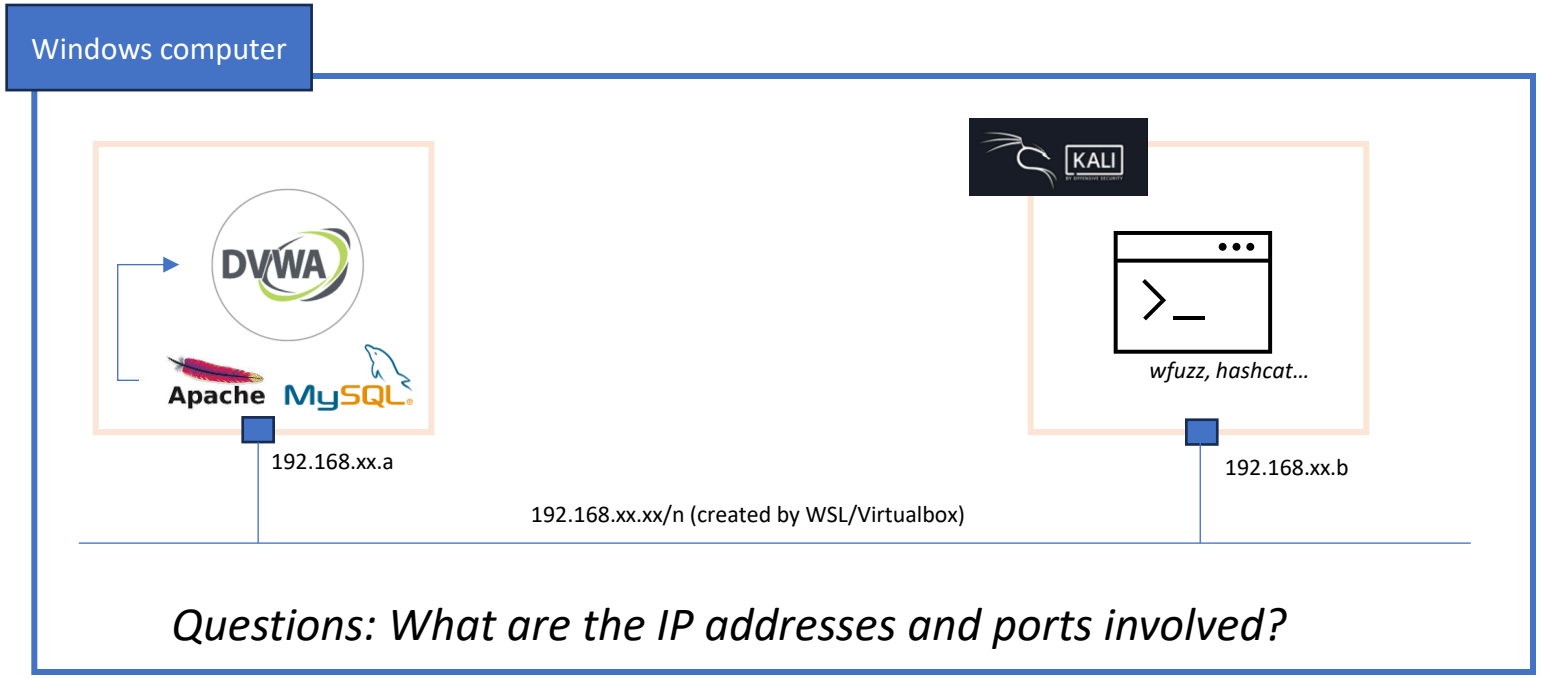
attack over the windows localhost with BurpSuite running in Windows



Strictly no localhost, no 127.0.0.1 !!!! Please redo the lab with IP address of the client and server.

Getting Pro

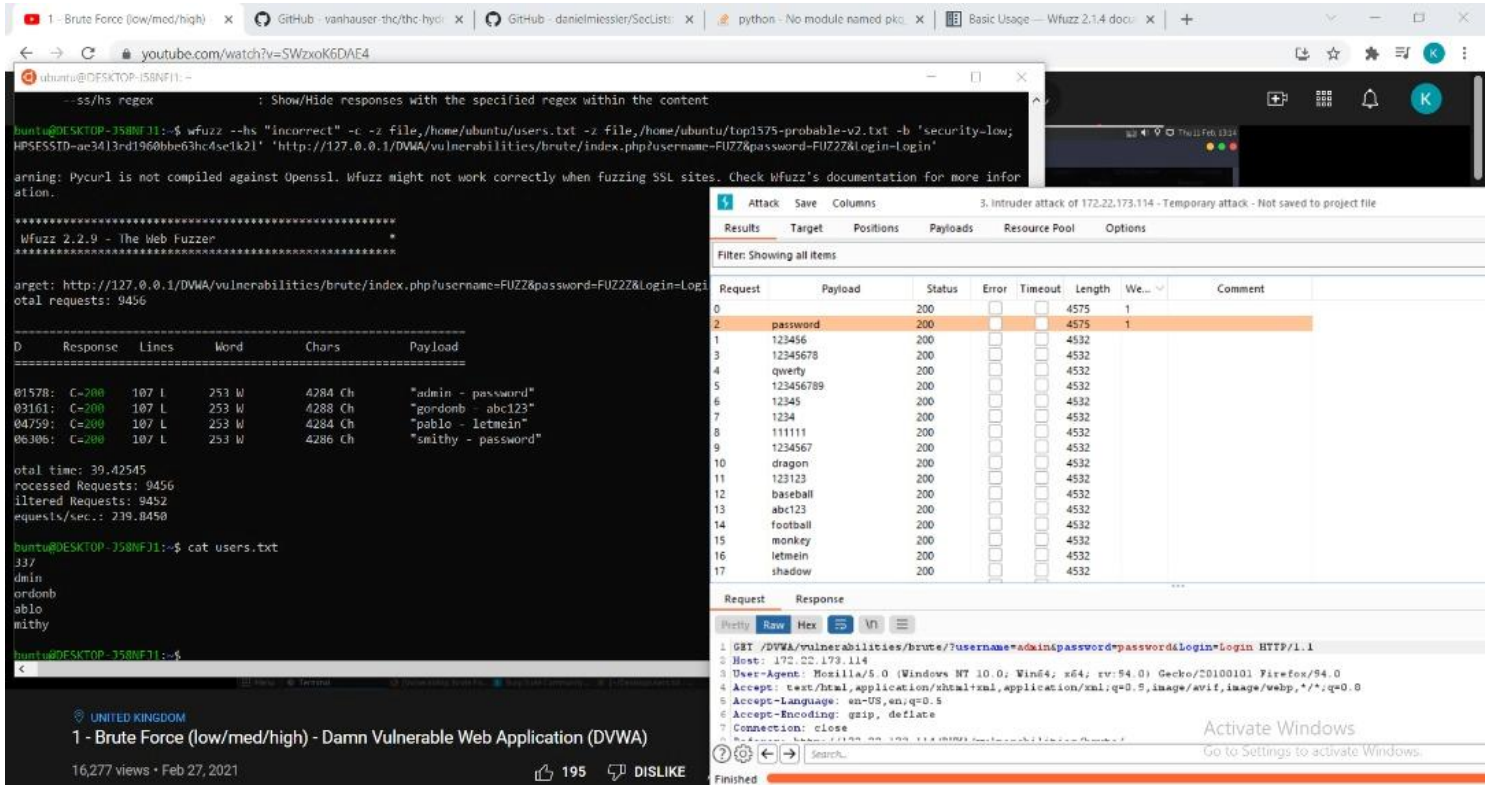
attack with tools in Kali Linux over the WSL (172.xx.xx.xx) or Virtualbox (192.168.56.xx) network



Step 1: Complete DVWA 01 lab - brute force attack

1.1 Using burp to brute force admin account
1.2 Using wfuzz to brute force all the user accounts
1.3 Repeat brute force attack with DVWA server set to ‘mid level security’. Analyze the HTTP traffic with tcpdump/wireshark
1.4 Repeat brute force attack with DVWA server set to ‘high level security’

Additional question: Configure firewall (OPNsense) to prevent brute force attack.



wfuzz --hs "incorrect" -c -z file,/home/ubuntu/users.txt -z file,/home/ubuntu/top1575-probable-v2.txt -b 'security=low; PHPSESSID=ae34l3rd1960bbe63hc4se1k2l' 'http://127.0.0.1/DVWA/vulnerabilities/brute/index.php?username=FUZZ&password=FUZ2Z&Login=Login’

Wireshark is your best tools!

Your lab report should reflect your understanding…

Please discuss with your lecturer before the lab to verify your understanding

Lab report layout for each vulnerability

kelemahan

a) What is the **vulnerability** and what will happen to the **CIA** when the vulnerability is exploited?

permukaan serangan

b) How the vulnerability can be exploited? What is the **attack surface**?

Bagaimana *attacker* dapat menyerang kelemahan itu

c) Wireshark network adapter(s) used, filtering used, and flow graph analysis of the attack.

d) Result and discussion from the captured wireshark packets.

Your lab report should reflect your understanding…

Please discuss with your lecturer before the lab to verify your understanding

Lab report layout for each vulnerability

kelemahan

a) What is the **vulnerability** and what will happen to the **CIA** when the vulnerability is exploited?

Permukaan serangan

b) How the vulnerability can be exploited? What is the **attack surface**?

c) Wireshark network adapter(s) used, filtering used, and flow graph analysis of the attack.

d) Result and discussion from the captured wireshark packets.

Bagaimana *attacker* dapat menyerang kelemahan itu

Dengan menggunakan Wireshark, tunjukkan cara penyerangan yang telah berlaku

Your lab journey…

1. Ping between windows and Linux computer.
2. Wireshark packet capture of ping activity in 1.
3. Apache web server setup and wireshark packet to show HTTP packets between the browser and Apache web server. Please explain the port number used.
4. DVWA setup and repeat wireshark packet capturing activities in 3.
5. Password brute force attack with burp suite and repeat wireshark packet capture activities.

Figure 1.2 displays all the information required to determine the host's logical and physical address, Terminal Ubuntu (172.29.2.251), using ifconfig.

```
C:\Users\Gigabyte>ping 172.29.2.251

Pinging 172.29.2.251 with 32 bytes of data:
Reply from 172.29.2.251: bytes=32 time<1ms TTL=64
Reply from 172.29.2.251: bytes=32 time<1ms TTL=64
Reply from 172.29.2.251: bytes=32 time<1ms TTL=64
Reply from 172.29.2.251: bytes=32 time<1ms TTL=64

Ping statistics for 172.29.2.251:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Figure 1.3: Ping 172.29.2.251 command in Command Prompt

```
bellis@DESKTOP-DVCR9IH:~$ ping 172.29.0.1
PING 172.29.0.1 (172.29.0.1) 56(84) bytes of data.
64 bytes from 172.29.0.1: icmp_seq=1 ttl=128 time=0.275 ms
64 bytes from 172.29.0.1: icmp_seq=2 ttl=128 time=0.454 ms
64 bytes from 172.29.0.1: icmp_seq=3 ttl=128 time=0.733 ms
64 bytes from 172.29.0.1: icmp_seq=4 ttl=128 time=0.500 ms
64 bytes from 172.29.0.1: icmp_seq=5 ttl=128 time=0.534 ms
64 bytes from 172.29.0.1: icmp_seq=6 ttl=128 time=0.329 ms
64 bytes from 172.29.0.1: icmp_seq=7 ttl=128 time=0.689 ms
64 bytes from 172.29.0.1: icmp_seq=8 ttl=128 time=0.639 ms
^C
--- 172.29.0.1 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7908ms
rtt min/avg/max/mdev = 0.275/0.519/0.733/0.153 ms
```

Figure 1.4: Ping 172.29.0.1 command in Ubuntu

2. Wireshark packet capture of ping activity in 1.

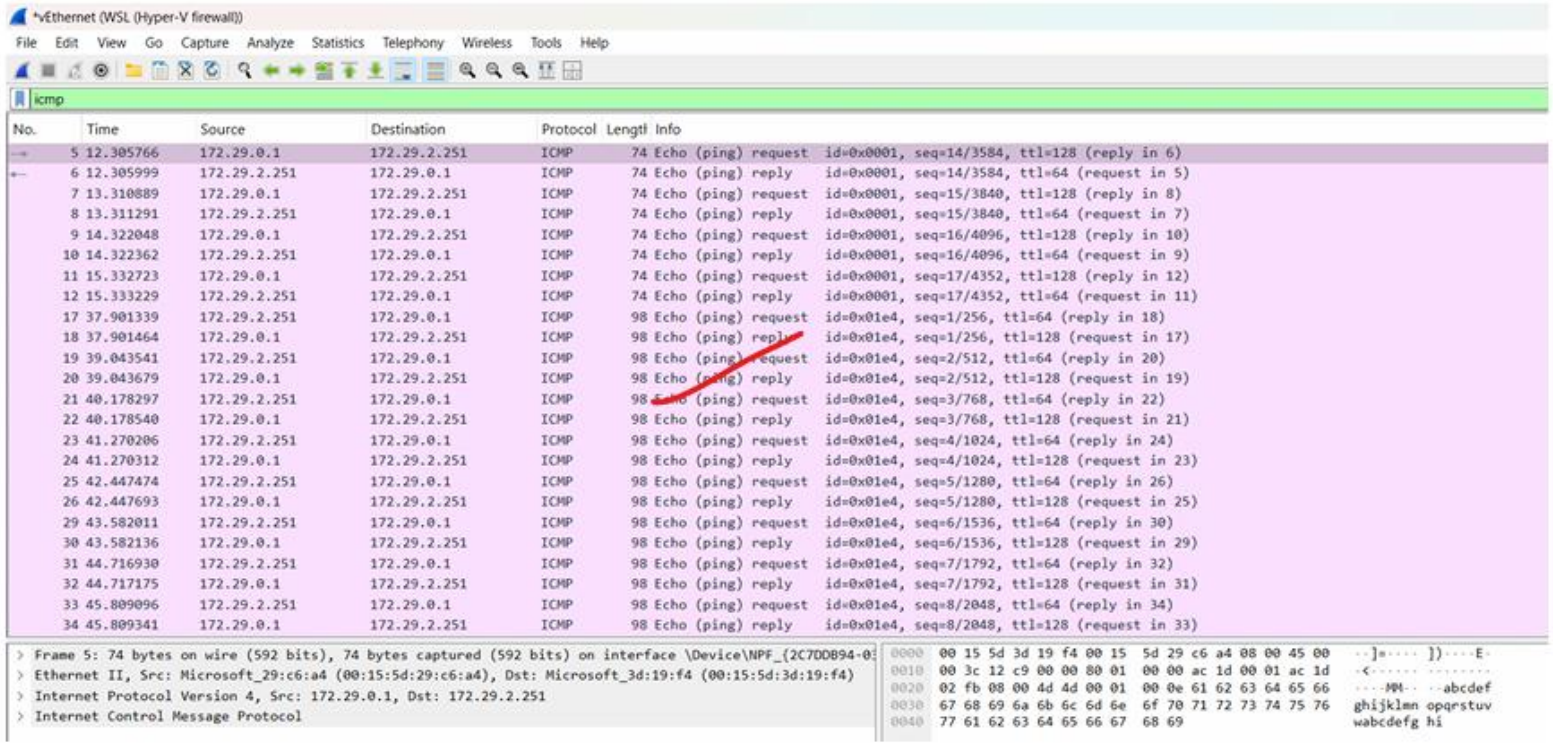


Figure 2.1: Wireshark packet capture of ping activity

Internet Control Message Protocol (ICMP) is a network protocol that allows devices to communicate errors and operational information when communicating with other IP addresses. Figure 2.1 shows Wireshark with an ICMP filter applied, displaying captured packets during the ping activity. The captured packets illustrate that the ICMP packets are successfully transmitted and received which proves there is network communication.

3. Apache web server setup and wireshark packet to show HTTP packets between the browser and Apache web server. Please explain the port number used.

```
bellis@DESKTOP-DVCR9IH:/$ sudo netstat -peanut
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       User       Inode      PID/Program name
tcp        0      0 10.255.255.254:53       0.0.0.0:*               LISTEN      0          20490      -
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      991        21103      105/systemd-resolve
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      106        50063      9479/mariadbd
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      991        21101      105/systemd-resolve
tcp6       0      0 :::80                   :::*                    LISTEN      0          55702      11207/apache2
udp        0      0 127.0.0.54:53           0.0.0.0:*                           991        21102      105/systemd-resolve
udp        0      0 127.0.0.53:53           0.0.0.0:*                           991        21100      105/systemd-resolve
udp        0      0 10.255.255.254:53       0.0.0.0:*                           0          20489      -
udp        0      0 127.0.0.1:323           0.0.0.0:*                           0          21511      -
udp6       0      0 ::1:323                 :::*                                0          21512      -
```

Figure 3.1: sudo netstat -peanut command in Ubuntu

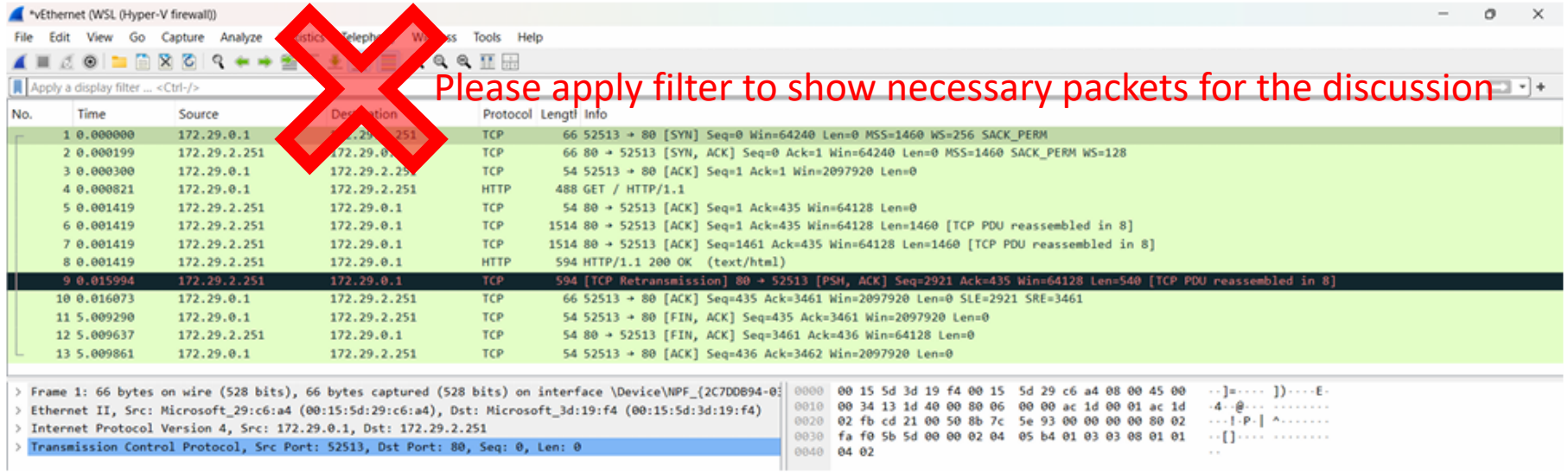


Figure 3.3: Wireshark packet capture of HTTP request and response

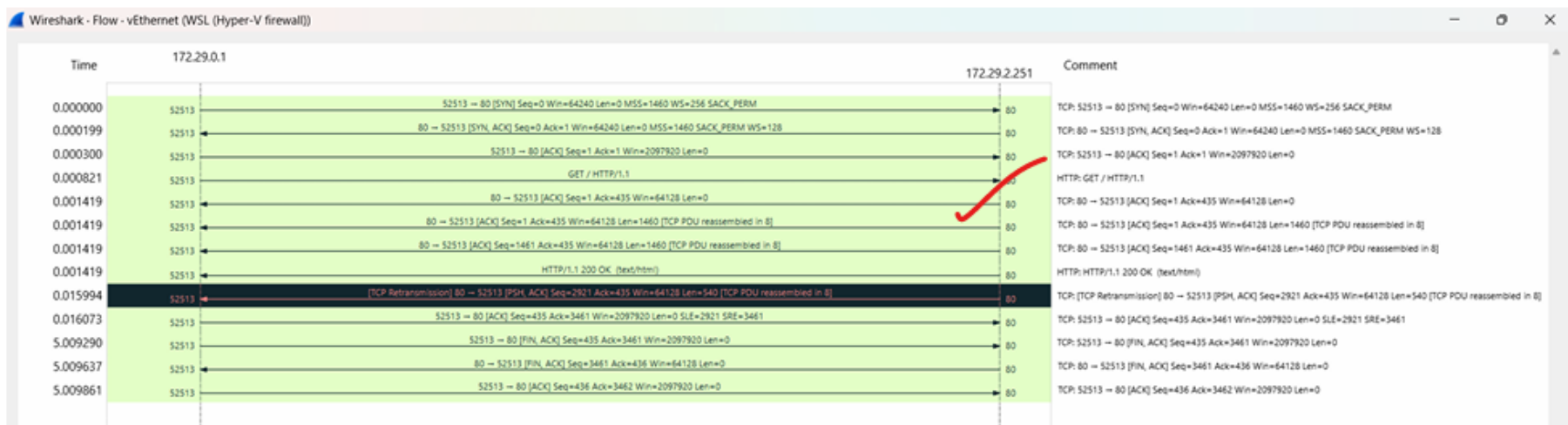


Figure 3.4: Flow graph

Capturing from vEthernet (WSL (Hyper-V firewall))

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
250	1.505687	172.29.0.1	172.29.2.251	HTTP	589	GET /DVWA/vulnerabilities/brute/?username=wong&password=fighting&Login=Login HTTP/1.1
253	1.511043	172.29.2.251	172.29.0.1	HTTP	352	HTTP/1.1 200 OK (text/html)
256	1.784012	172.29.0.1	172.29.2.251	HTTP	586	GET /DVWA/vulnerabilities/brute/?username=12345&password=this&Login=Login HTTP/1.1
258	1.790768	172.29.2.251	172.29.0.1	HTTP	351	HTTP/1.1 200 OK (text/html)
264	2.067033	172.29.0.1	172.29.2.251	HTTP	585	GET /DVWA/vulnerabilities/brute/?username=1234&password=this&Login=Login HTTP/1.1
267	2.074619	172.29.2.251	172.29.0.1	HTTP	352	HTTP/1.1 200 OK (text/html)
270	2.390497	172.29.0.1	172.29.2.251	HTTP	584	GET /DVWA/vulnerabilities/brute/?username=123&password=this&Login=Login HTTP/1.1
272	2.393453	172.29.2.251	172.29.0.1	HTTP	351	HTTP/1.1 200 OK (text/html)
278	2.703537	172.29.0.1	172.29.2.251	HTTP	583	GET /DVWA/vulnerabilities/brute/?username=12&password=this&Login=Login HTTP/1.1
281	2.710948	172.29.2.251	172.29.0.1	HTTP	352	HTTP/1.1 200 OK (text/html)
284	3.014911	172.29.0.1	172.29.2.251	HTTP	582	GET /DVWA/vulnerabilities/brute/?username=1&password=this&Login=Login HTTP/1.1
286	3.049456	172.29.2.251	172.29.0.1	HTTP	351	HTTP/1.1 200 OK (text/html)
292	3.392848	172.29.0.1	172.29.2.251	HTTP	586	GET /DVWA/vulnerabilities/brute/?username=admin&password=this&Login=Login HTTP/1.1
295	3.400283	172.29.2.251	172.29.0.1	HTTP	352	HTTP/1.1 200 OK (text/html)
298	3.823593	172.29.0.1	172.29.2.251	HTTP	589	GET /DVWA/vulnerabilities/brute/?username=notadmin&password=this&Login=Login HTTP/1.1
300	3.826415	172.29.2.251	172.29.0.1	HTTP	351	HTTP/1.1 200 OK (text/html)
306	4.295734	172.29.0.1	172.29.2.251	HTTP	585	GET /DVWA/vulnerabilities/brute/?username=what&password=this&Login=Login HTTP/1.1
309	4.298278	172.29.2.251	172.29.0.1	HTTP	352	HTTP/1.1 200 OK (text/html)
312	4.602393	172.29.0.1	172.29.2.251	HTTP	585	GET /DVWA/vulnerabilities/brute/?username=when&password=this&Login=Login HTTP/1.1

```
> Frame 292: 586 bytes on wire (4688 bits), 586 bytes captured (4688 bits) on interface \Device\NPF_{92D3BC05-D78D-4A93-948B-DC65C8B4CBB1}, id 0
> Ethernet II, Src: Microsoft_16:f2:40 (00:15:5d:16:f2:40), Dst: Microsoft_02:28:74 (00:15:5d:02:28:74)
> Internet Protocol Version 4, Src: 172.29.0.1, Dst: 172.29.2.251
> Transmission Control Protocol, Src Port: 49367, Dst Port: 80, Seq: 1, Ack: 1, Len: 532
v Hypertext Transfer Protocol
  > GET /DVWA/vulnerabilities/brute/?username=admin&password=this&Login=Login HTTP/1.1\r\n
    Host: 172.29.2.251\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:133.0) Gecko/20100101 Firefox/133.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate, br\r\n
    Connection: keep-alive\r\n
    Referer: http://172.29.2.251/DVWA/vulnerabilities/brute/\r\n
  > Cookie: security=low; PHPSESSID=ihhri1i6nf3fomkspmbtb3d4os\r\n
    Upgrade-Insecure-Requests: 1\r\n
    Priority: u=0, i\r\n
    \r\n
```

```
0000  00 15 5d 02 28 74 00 15  5d 16 f2 40 08 00 45 00   ··]·
0010  02 3c f2 8f 40 00 80 06  00 00 ac 1d 00 01 ac 1d   ·<··
0020  02 fb c0 d7 00 50 a9 87  f5 61 fb 7a 63 bd 50 18   ····
0030  04 02 5d 65 00 00 47 45  54 20 2f 44 56 57 41 2f   ··]e
0040  76 75 6c 6e 65 72 61 62  69 6c 69 74 69 65 73 2f   vuln
0050  62 72 75 74 65 2f 3f 75  73 65 72 6e 61 6d 65 3d   brut
0060  61 64 6d 69 6e 26 70 61  73 73 77 6f 72 64 3d 74   admi
0070  68 69 73 26 4c 6f 67 69  6e 3d 4c 6f 67 69 6e 20   his&
0080  48 54 54 50 2f 31 2e 31  0d 0a 48 6f 73 74 3a 20   HTTP
0090  31 37 32 2e 32 39 2e 32  2e 32 35 31 0d 0a 55 73   172.
00a0  65 72 2d 41 67 65 6e 74  3a 20 4d 6f 7a 69 6c 6c   er-A
00b0  61 2f 35 2e 30 20 28 57  69 6e 64 6f 77 73 20 4e   a/5.
00c0  54 20 31 30 2e 30 3b 20  57 69 6e 36 34 3b 20 78   T 10
00d0  36 34 3b 20 72 76 3a 31  33 33 2e 30 29 20 47 65   64;
00e0  63 6b 6f 2f 32 30 31 30  30 31 30 31 20 46 69 72   cko/
00f0  65 66 6f 78 2f 31 33 33  2e 30 0d 0a 41 63 63 65   efox
0100  70 74 3a 20 74 65 78 74  2f 68 74 6d 6c 2c 61 70   pt:
0110  70 6c 69 63 61 74 69 6f  6e 2f 78 68 74 6d 6c 2b   plic
0120  78 6d 6c 2c 61 70 70 6c  69 63 61 74 69 6f 6e 2f   xml,
0130  78 6d 6c 3b 71 3d 30 2e  39 2c 2a 2f 2a 3b 71 3d   xml;
```

Figure 5.16: Wireshark packet capture of a cluster bomb attack at a low security level with HTTP filter

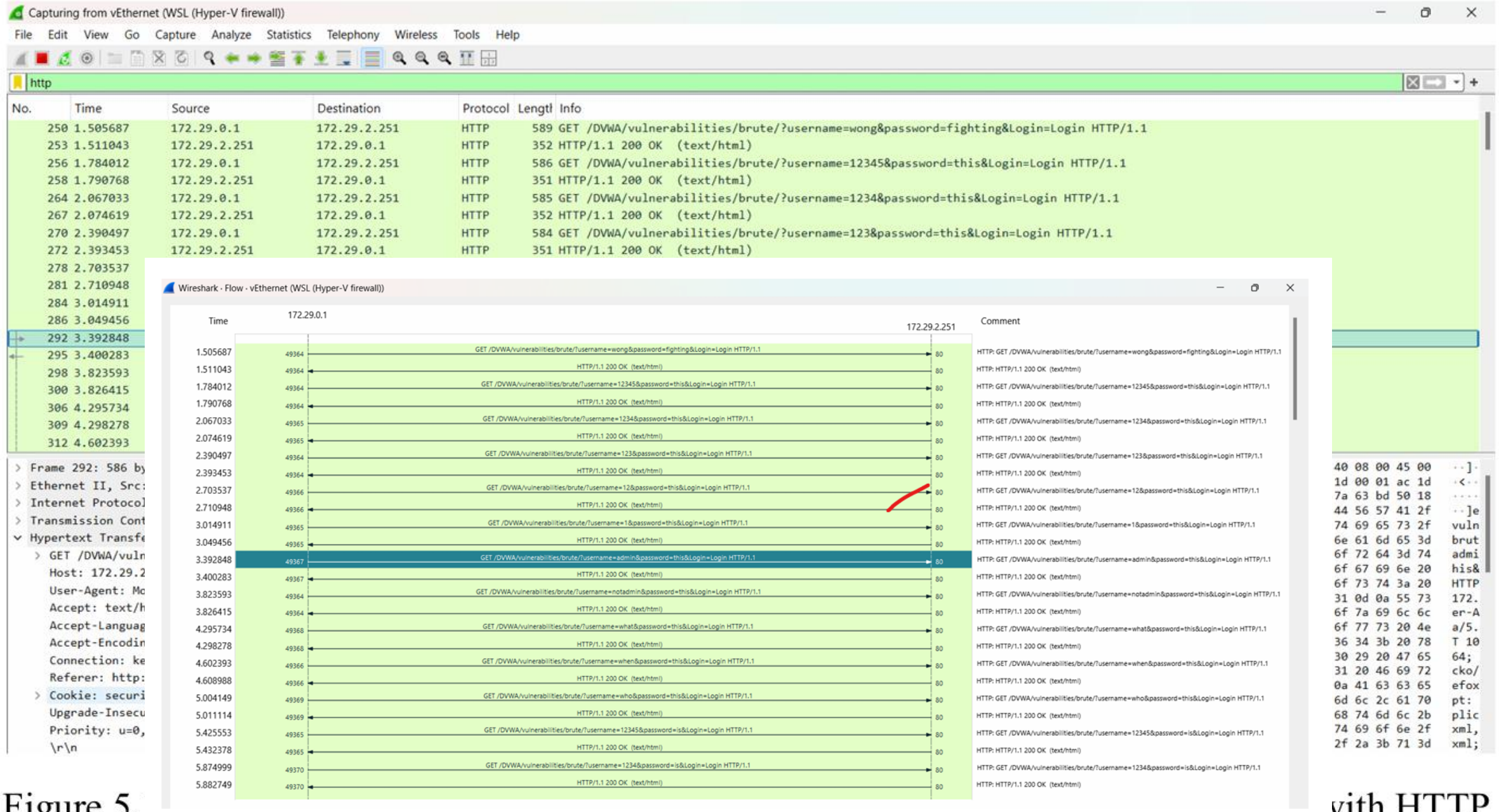


Figure 5. vith HTTP

Figure 5.17: Flow graph

Wireshark · Conversations · vEthernet (WSL (Hyper-V firewall))

Conversation Settings

- [ ] Name resolution
- [x] Absolute start time
- [x] Limit to display filter

Copy

Follow Stream...

Graph...

- [ ] Protocol
- [ ] Bluetooth
- [ ] BPv7
- [ ] DCCP
- [x] Ethernet
- [ ] FC
- [ ] FDDI
- [ ] IEEE 802.11
- [ ] IEEE 802.15.4
- [x] IPv4
- [x] IPv6
- [ ] IPX
- [ ] JXTA
- [ ] LTP
- [ ] MPTCP
- [ ] NCP
- [ ] openSAFETY
- [ ] RSVP
- [ ] SCTP

Filter list for specific type

Ethernet · 1 | IPv4 · 1 | IPv6 | TCP · 25 | UDP

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Abs Start	Duration	Bits/s A → B	Bits/s B → A	Flows
172.29.0.1	64453	172.29.2.251	80	3	162 bytes	0	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.394	0.0006			0
172.29.0.1	64454	172.29.2.251	80	3	162 bytes	6	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0002			0
172.29.0.1	64455	172.29.2.251	80	3	162 bytes	8	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0002			0
172.29.0.1	64456	172.29.2.251	80	3	162 bytes	7	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0004			0
172.29.0.1	64457	172.29.2.251	80	3	162 bytes	5	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0003			0
172.29.0.1	64458	172.29.2.251	80	3	162 bytes	4	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0005			0
172.29.0.1	64459	172.29.2.251	80	3	162 bytes	9	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0002			0
172.29.0.1	64460	172.29.2.251	80	3	162 bytes	3	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0005			0
172.29.0.1	64461	172.29.2.251	80	3	162 bytes	2	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.395	0.0004			0
172.29.0.1	64462	172.29.2.251	80	3	162 bytes	1	3	100.00%	2	108 bytes	1	54 bytes	21:15:04.394	0.0004			0
172.29.0.1	64495	172.29.2.251	80	31	15 kB	10	31	100.00%	13	4 kB	18	11 kB	21:15:04.723	10.6957	2738 bits/s	8419 bits/s	10
172.29.0.1	64496	172.29.2.251	80	24	10 kB	11	24	100.00%	11	3 kB	13	7 kB	21:15:05.235	8.5532	2758 bits/s	6433 bits/s	7
172.29.0.1	64498	172.29.2.251	80	21	9 kB	12	21	100.00%	9	2 kB	12	7 kB	21:15:05.798	11.6772	1551 bits/s	4675 bits/s	6
172.29.0.1	64499	172.29.2.251	80	16	6 kB	13	16	100.00%	7	2 kB	9	5 kB	21:15:06.499	8.0291	1562 bits/s	4591 bits/s	4
172.29.0.1	64500	172.29.2.251	80	16	6 kB	14	16	100.00%	7	2 kB	9	5 kB	21:15:07.121	9.2787	1351 bits/s	3972 bits/s	4
172.29.0.1	64501	172.29.2.251	80	16	6 kB	15	16	100.00%	7	2 kB	9	5 kB	21:15:07.908	10.2561	1221 bits/s	3594 bits/s	4
172.29.0.1	64502	172.29.2.251	80	11	3 kB	16	11	100.00%	5	871 bytes	6	2 kB	21:15:08.729	5.3892	1292 bits/s	3550 bits/s	2
172.29.0.1	64503	172.29.2.251	80	11	3 kB	17	11	100.00%	5	871 bytes	6	2 kB	21:15:09.650	5.3167	1310 bits/s	3599 bits/s	2
172.29.0.1	64504	172.29.2.251	80	11	3 kB	18	11	100.00%	5	871 bytes	6	2 kB	21:15:10.668	5.2290	1332 bits/s	3659 bits/s	2
172.29.0.1	64505	172.29.2.251	80	11	3 kB	19	11	100.00%	5	871 bytes	6	2 kB	21:15:11.788	5.1367	1356 bits/s	3725 bits/s	2
172.29.0.1	64506	172.29.2.251	80	11	3 kB	20	11	100.00%	5	871 bytes	6	2 kB	21:15:15.788	5.0200	1388 bits/s	3811 bits/s	2
172.29.0.1	64508	172.29.2.251	80	11	3 kB	21	11	100.00%	5	871 bytes	6	2 kB	21:15:16.504	5.0379	1383 bits/s	3798 bits/s	2
172.29.0.1	64509	172.29.2.251	80	11	3 kB	22	11	100.00%	5	871 bytes	6	2 kB	21:15:17.276	5.1099	1363 bits/s	3744 bits/s	2
172.29.0.1	64510	172.29.2.251	80	11	3 kB	23	11	100.00%	5	871 bytes	6	2 kB	21:15:18.075	5.1746	1346 bits/s	3698 bits/s	2
172.29.0.1	64511	172.29.2.251	80	11	3 kB	24	11	100.00%	5	870 bytes	6	2 kB	21:15:18.898	5.2549	1324 bits/s	3717 bits/s	2

How many TCP connections to the server per password brute force attack launch with burpsuite/hydra?

Figure 5.28: Wireshark conversation

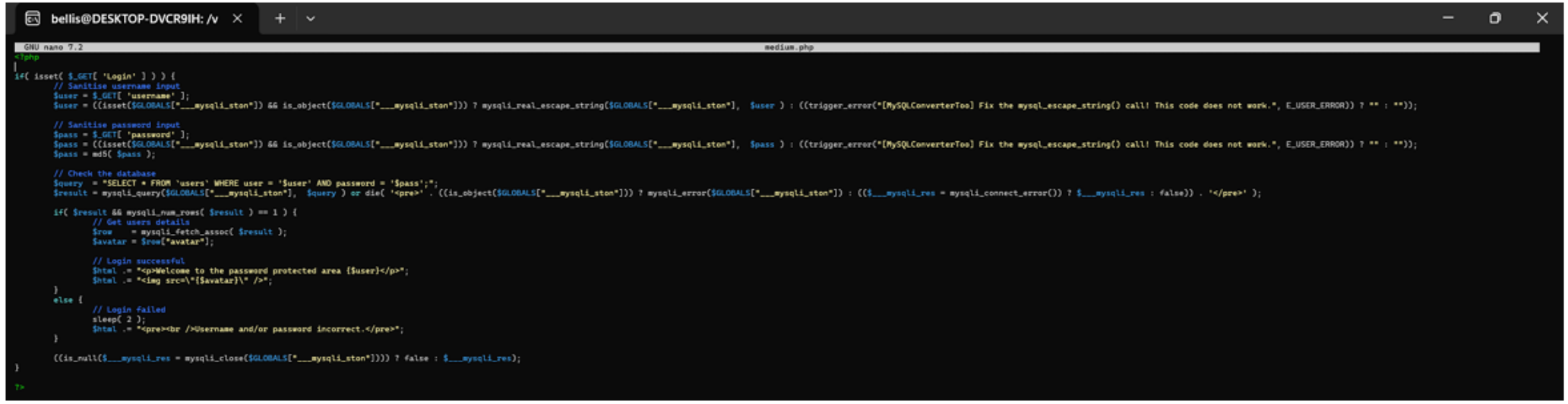


Figure 5.35: medium.php file

PHP source code for medium-level security:

```
<?php


if( isset( $_GET[ 'Login' ] ) ) {
        // Sanitise username input
        $user = $_GET[ 'username' ];
        $user = ((isset($GLOBALS["___mysqli_ston"]) &&
is_object($GLOBALS["___mysqli_ston"])) ?
mysqli_real_escape_string($GLOBALS["___mysqli_ston"],  $user ) :
```

Discuss the source code